

محرك المعالجة عالي الأداء لنظام التجارة الإلكترونية

High-Performance E-Commerce Backend Engine

تقرير مشروع مادة البرمجة المتوازية — الفصل الدراسي 2026

فهرس المحتويات

١. نظرة عامة على المشروع

٢. معمارية النظام (System Architecture)

٣. NFR-1: حماية البيانات المشتركة من التضرار

٤. NFR-2: إدارة الموارد الحاسوبية

٥. NFR-3: المعالجة غير المتزامنة (Async Queues)

٦. NFR-4: معالجة البيانات الضخمة على دفعات (Batch Processing)

٧. NFR-5: توزيع الأحمال (Load Distribution)

٨. NFR-6: التخزين المؤقت الموزع (Distributed Caching)

٩. NFR-7: التحكم في الأقفال (Concurrency Control)

١٠. NFR-8: سلامة المعاملات (ACID Transactions)

١١. NFR-9: اختبار الاستقرار تحت الضغط (Stress Testing)

١٢. NFR-10: القياس وتحديد الاختناقات (Benchmarking)

١٣. تحسينات إضافية — ما وراء المتطلبات

١٤. جدول شامل — كل نقاط التزامن في التطبيق

١٥. الخاتمة

١. نظرة عامة على المشروع

هذا المشروع هو محرك معالجة عالي الأداء لمنظومة تجارة إلكترونية تم بناؤه باستخدام NestJS v11 مع PostgreSQL و Redis و BullMQ. الهدف الأساسي ليس مجرد بناء متجر إلكتروني عادي، بل **تطبيق مفاهيم البرمجة المتوازية والأنظمة الموزعة** التي تمت دراستها في المادة: التزامن، التزامن العددي، الأقفال الموزعة، المعالجة غير المتزامنة، توزيع الأحمال، التخزين المؤقت، وسلامة المعاملات في بيئة متعددة الخيوط.

التطبيق يعمل على **3 نسخ (instances)** متزامنة في Docker Compose خلف Nginx كوكيل عكسي. هذا يعني أن 3 عمليات من NestJS تعمل في نفس الوقت، وكل طلب يمكن أن يذهب إلى أي نسخة. التحدي الحقيقي هو: كيف نضمن أن هذه النسخ الثلاث تتصرف وكأنها نظام واحد متكامل دون تضارب في البيانات؟

التقنيات المستخدمة

المكوّن	التقنية	الغرض
إطار العمل	NestJS v11 (Node.js / TypeScript)	تطبيق معياري منظم يعتمد على Dependency Injection و Decorators
قاعدة البيانات	PostgreSQL 15 + TypeORM	بيانات العلائقية مع دعم pessimistic locks والمعاملات (ACID)
التخزين المؤقت والتنسيق	Redis 8 (ioredis)	أقفال موزعة، سيمافور، عدادات ذرية، تغذية BullMQ
قوائم الانتظار	BullMQ	مهام غير متزامنة (إيميل، فواتير، تنبيهات، معالجة دفعات)
الوكيل العكسي	Nginx 1.25	توزيع الأحمال (Round-Robin)، failover، مولة 30s
التخزين	MinIO (S3-compatible)	تخزين فواتير PDF وتقارير المبيعات
المراقبة	Prometheus + Grafana	قياس الأداء، لوحات معلومات، تتبع الاختناقات
الاختبار	k6 + Node.js scripts	اختبار إجهاد، 6 سيناريوهات تزامن مختلفة

ملاحظة مهمة: في الوقت الحالي، التطبيق يعمل على 3 نسخ. ولكن تم تصميم كل آليات التزامن باستخدام

٢. معمارية النظام (System Architecture)

٢.١ المكونات الرئيسية

النظام مبني على طبقات واضحة، كل طبقة مسؤولة عن جانب محدد. هذا التقسيم يسمح بعزل نقاط التزامن وتطبيق آليات مختلفة لكل طبقة:

الطبقة	المكونات	دورها في التزامن
API Layer	Controllers, Guards, Interceptors, DTOs, Pipes	تستقبل الطلبات وتُمرّرها للخدمات. هنا يتم تطبيق Rate Limiting (5 طلبات/60 ثانية) لمنع الهجمات المبسطة
Service Layer	AuthService, CartService, WalletService, InventoryService	منطق الأعمال. هنا تبدأ نقاط التزامن الحقيقية — أقفال Redis، أقفال قاعدة البيانات، إلخ
CheckoutFacade	واجهة موحّدة لعملية الشراء	أهم نقطة تزامن في النظام. تنسّق: قفل المستخدم ← سيمافور المخزون ← معاملة قاعدة البيانات ← إطلاق المهام غير المتزامنة
Queue Layer	BullMQ Workers: Email, Invoice, Low-Stock, Batch	تنفيذ المهام غير المتزامنة مع تحديد عدد المهام المتزامنة (concurrency) لكل معالج
Persistence	PostgreSQL + TypeORM	المصدر النهائي للحقيقة. pessimistic locks + ACID transactions تضمن عدم فقدان البيانات أو ازدواجيتها
Coordination	Redis + DistributedLockService	قلب النظام الموّجّع. أقفال حصرية، سيمافور، عدادات ذرية — كلها في Redis حتى تراها جميع النسخ

٢.٢ نمط الواجهة الموحّدة (Facade Pattern)

CheckoutFacade هو نبض النظام. كل عملية شراء تمر عبره، وهو ينسّق 6 خطوات على الأقل:

1. التحقق من idempotency — إذا كان الطلب مكرراً، نعيد النتيجة السابقة
2. قفل المستخدم الحصري — lock: checkout: {userId} في Redis
3. التحقق من السلة — ليست فارغة، المنتجات موجودة
4. سيمافور المخزون — sem: stock: {productId} في Redis، عدد المسموح به = المخزون الفعلي
5. معاملة قاعدة البيانات — قفل Pessimistic Write على المستخدم والمنتج، خصم، إنشاء الطلب
6. إطلاق المهام غير المتزامنة — إيميل، فاتورة، تنبيه المخزون

٢.٣ تطبيق AOP (البرمجة الموجهة للجوانب) — Interceptors + Winston 3

AOP يعني أننا نعزل الوظائف المشتركة (Cross-cutting Concerns) مثل التسجيل والمراقبة وقياس الأداء في طبقة منفصلة، بعيداً عن منطق الأعمال. في هذا المشروع، طبقنا AOP عبر **Interceptors 3** عالميين — كل **Interceptor** مسؤول عن جانب مختلف — **بدون أي تعديل في كود الخدمات أو الـ Controllers**.

التقنية	المسؤولية	الـ Interceptor
LoggerService (Winston) + ملفات Console + (JSON)	تسجيل كل طلب HTTP مع بيانات كاملة: المستخدم، الـ IP، الـ Body، زمن التنفيذ، والاستجابة — كل في ملف مخصص حسب الـ Controller والـ Handler	LoggingInterceptor
Prometheus + prom-client + Grafana	جمع مقاييس Prometheus: عدد الطلبات (Counter) وزمن التنفيذ (Histogram) مصنفة حسب status + route + method code	MetricsInterceptor
NestJS Logger	تسجيل بسيط لزمن التنفيذ (method + url + duration) في Console للتصحيح السريع	PerformanceInterceptor

LoggingInterceptor — التسجيل التفصيلي لكل Endpoint

```
// src/common/interceptors/logging.interceptor.ts — حقيقي AOP
@Injectable()
export class LoggingInterceptor implements NestInterceptor {
  constructor(private readonly loggerService: LoggerService) {}

  intercept(context: ExecutionContext, next: CallHandler): Observable<any> {
    const request = context.switchToHttp().getRequest();
```

```

const controller = context.getClass().name;
const handler = context.getHandler().name;
const method = request.method;
const url = request.originalUrl ?? request.url;
const userId = request.user?.userId;
const userRole = request.user?.role;
const ip = request.ip ?? request.headers?.['x-forwarded-for'];
const requestBody = method !== 'GET' ? request.body : undefined;
const startedAt = performance.now();

return next.handle().pipe(
  tap({
    next: () => {
      const duration = Math.round(performance.now() - startedAt);
      this.loggerService.logControllerAction(
        controller, handler, method, url,
        response.statusCode, duration,
        userId, userRole, ip, userAgent, requestBody,
      );
    },
    error: (error) => {
      // يسجل الأخطاء أيضاً مع كامل البيانات
      this.loggerService.logControllerAction(
        controller, handler, method, url,
        error.status ?? 500, duration,
        userId, userRole, ip, userAgent, requestBody, error.message,
      );
    },
  }),
);
}
}

```

Endpoint مع ملفات منفصلة لكل Winston LoggerService

هذا هو قلب نظام التسجيل. مبني على Winston مع 3 مخرجات:

- **Console**: ألوان وتنسيق readable للتطوير
- **combined.log**: كل الـ logs (بنظام JSON) مع Rotate (حتى 5 ملفات × 10MB)
- **error.log**: الأخطاء فقط (مستوى error) مع Rotate

- **ملفات لكل log Endpoint:** logs/{controller}/{handler}_logs.log — كل handler له ملف
JSON منفصل (حتى 3 ملفات × 5MB). مثلاً: logs/products/findAll_logs.log
logs/orders/checkout_logs.log

```
// src/logger/logger.service.ts
private getEndpointLogger(controller: string, handler: string):
winston.Logger {
  const folder = controller.replace(/Controller$/i, '').toLowerCase();
  const fileName = `${handler}_logs.log`;
  // ينشئ ملف : logs/{folder}/{fileName}
  // مثل : logs/products/findAll_logs.log
}

logControllerAction(controller, handler, method, url, statusCode, duration,
  userId?, userRole?, ip?, userAgent?, requestBody?,
responseBody?) {
  // endpoint مخصص ملف الـ Console + combined.log : يسجل في 3 أماكن
}
```

Prometheus — MetricsInterceptor للوحة القيادة

```
// src/common/interceptors/metrics.interceptor.ts
const httpRequestDuration = new promClient.Histogram({
  name: 'http_request_duration_seconds',
  labelNames: ['method', 'route', 'status_code'],
  buckets: [0.01, 0.05, 0.1, 0.25, 0.5, 1, 2.5, 5, 10],
});

const httpRequestsTotal = new promClient.Counter({
  name: 'http_requests_total',
  labelNames: ['method', 'route', 'status_code'],
});
```

هذا الـ **Interceptor** يجمع مقاييس Prometheus كل 15 ثانية ويعرضها في Grafana (على 3001 port). اللوحة تظهر: عدد الطلبات في الثانية، زمن الاستجابة (P50/P95/P99)، ومعدل الأخطاء — كلها مصنفة حسب

٣. حماية البيانات المشتركة من التضارب (Concurrent Access & Data Integrity)

٣.١ المشكلة

تخيل أن لديك منتجاً في المخزون $quantity = 1$. في نفس اللحظة، 100 مستخدم يضغطون على "اشتر الآن". إذا لم تكن هناك آلية تزامن، النظام سيقراً المخزون 100 مرة (كل مرة يرى 1)، ويوافق على 100 عملية شراء، ويصبح المخزون -99. هذه مشكلة **Race Condition** كلاسيكية — وتسمى "Overselling".

نفس المشكلة تحدث مع المحفظة: إذا حاول مستخدم إيداع \$50 مرتين بسرعة، بدون تزامن قد يقرأ النظام الرصيد القديم مرتين ويضيف \$50 مرتين على نفس الرقم الأساسي — تخسر \$50 (Lost Update).

٣.٢ الحل — 5 آليات متداخلة

آلية 1 — السيمافور الموزّع (Redis Semaphore)

قبل الوصول إلى قاعدة البيانات، يمر كل طلب شراء عبر سيمافور في Redis. السيمافور يسمح بعدد من الطلبات المتزامنة يساوي المخزون الفعلي. إذا المخزون = 50، أول 50 مشترٍ يدخلون فوراً، والـ 50 الباقون يُرفضون فوراً برسالة "المخزون غير متوفر".

ملاحظة مهمة: قيمة الخصم من السيمافور = **الكمية المطلوبة من المنتج** وليس 1. مثلاً إذا مستخدم يريد 3 وحدات من منتج X، السيمافور يخصم 3. هذا يمنع مشكلة **Overselling بالكمية** (مثلاً 10 وحدات مخزون، 8 مستخدمين كل منهم يريد 2 — السيمافور يسمح فقط لأول 5 لأن $10=2 \times 5$).

```
// Redis عملية ذرية واحدة في DistributedLockService في Lua script
-- KEYS[1] = sem:stock:{productId}
-- ARGV[1] = المخزون الأولي للهيئة
-- ARGV[2] = مدة الصلاحية بالمللي ثانية
-- ARGV[3] = الكمية المطلوبة (عدد الوحدات)
```



```

redis.call("SET", KEYS[1], ARGV[1], "PX", ARGV[2], "NX")
local remaining = redis.call("DECRBY", KEYS[1], ARGV[3])
if remaining >= 0 then
    redis.call("PEXPIRE", KEYS[1], ARGV[2])
    return 1  -- تم الحجز
else
    redis.call("INCRBY", KEYS[1], ARGV[3])
    return 0  -- المخزون أو الكمية غير كافية
end

```

آلية 2 — القفل التشاؤمي على المنتج (Postgres Pessimistic Write)

داخل معاملة قاعدة البيانات، كل منتج يُقفل بـ `SELECT ... FOR UPDATE`. هذا يضمن أنه حتى في السيمافور سمح بدخول أكثر من العدد المسموح (مثلاً بسبب إعادة التخزين بين خطوة السيمافور والمعاملة)، فإن قاعدة البيانات تمنع البيع الزائد.

```

// src/inventory/services/stock-validation.service.ts
const product = await manager.getRepository(Product).findOne({
  where: { id: productId },
  lock: { mode: "pessimistic_write" }, -- SELECT ... FOR UPDATE
});

```

آلية 3 — القفل التشاؤمي على المستخدم

في نفس المعاملة، يتم قفل صف المستخدم أيضاً لمنع أي عملية أخرى من تعديل رصيده في نفس الوقت:

```

// src/orders/facades/checkout.facade.ts — داخل dataSource.transaction
const user = await manager.getRepository(User).findOne({
  where: { id: userId },
  lock: { mode: "pessimistic_write" },
});

```

آلية 4 — الترتيب الثابت للأقفال (Lock Ordering)

لمنع **Deadlocks** — حالة تنتظر فيها معاملتان بعضهما بشكل دائري — يتم ترتيب المنتجات أبجدياً (UUID) قبل قفلها. هذا يضمن أن جميع المعاملات تكتسب الأقفال بنفس الترتيب، مما يمنع deadlock تماماً:

```
cartItems.sort((a, b) => a.productId.localeCompare(b.productId));
```

آلية 5 — القفل المتفائل على المنتج (TypeORM @VersionColumn)

بالإضافة إلى الأقفال التشاؤمية، كل منتج يحتوي على عمود `version` يتم زيادته تلقائياً مع كل تحديث عبر `@VersionColumn()` في TypeORM. هذا يوفر **طبقة دفاع إضافية (Defense-in-Depth)**:

- **في تدفق الشراء:** `FOR UPDATE` يضمن عدم وجود تعديل متزامن، لذا فحص الـ `version` يمرّ دائماً — وهو غير ضروري هنا ولكن لا يضر.
- **في مسارات أخرى** (مثل تعديل المشرف للمنتج أو `ProductsService.update()`): إذا حدث تضارب بين عمليّتي تعديل، `@VersionColumn()` يمنع **Lost Update** عن طريق رفض التحديث إذا كان الـ `version` مختلفاً.

```
// src/database/entities/product.entity.ts
@VersionColumn()
version: number;

// يُنشئ تلقائياً TypeORM:
// UPDATE products SET ..., version = version + 1
// WHERE id = ? AND version = ?
```

٣.٣ نتائج الاختبار

تم اختبار 100 مستخدم يحاولون شراء منتج بمخزون = 1 في نفس اللحظة:

المقياس	النتيجة المتوقعة	النتيجة الفعلية	الحالة
طلبات ناجحة (201)	1	1	✓
طلبات مرفوضة (400)	99	99	✓
أخطاء سيرفر (500)	0	0	✓
مخزون سالب (Overselling)	مستحيل	لم يحدث	✓

٤. إدارة الموارد الحاسوبية (Resource Management & Capacity Control)

٤.١ المشكلة

بدون تحديد عدد العمليات المتزامنة، يمكن للنظام أن يستهلك كل الذاكرة والمعالج تحت الضغط وينهار. كل نسخة من التطبيق محدودة بـ 1 CPU و 256MB RAM في Docker — وهذا يعني أننا لا نستطيع فتح عدد لا نهائي من الاتصالات المتزامنة أو معالجة عدد لا نهائي من الطلبات في نفس الوقت.

٤.٢ آليات إدارة الموارد — 4 مستويات

المستوى 1: السيمافور الموزع على المخزون (شرحنا في NFR-1)

السيمافور في Redis لا يحمي البيانات فقط — بل يدير الموارد أيضاً. بتحديد أن فقط N مشترٍ يمكنهم الدخول (حيث N = المخزون)، نحن نتحكم في عدد العمليات الثقيلة (معاملات قاعدة البيانات) التي ستنفذ بالتوازي. بدون السيمافور، 1000 مشترٍ على منتج بمخزون 50 سيخلقون 1000 معاملة — DB 950 منها ستفشل ولكن بعد استخدام موارد ثمينة.

المستوى 2: BullMQ Concurrency Limits

كل طابور في BullMQ له حد أقصى لعدد المهام المتزامنة لكل نسخة. هذا يمنع النظام من الانهيار تحت وطأة مئات الإيميلات أو الفواتير المتزامنة:

الطابور	الحد الأقصى للتزامن	قابل للتعديل؟
EMAIL_QUEUE	5	نعم (متغير بيئة)
INVOICE_QUEUE	3	نعم (متغير بيئة)
LOW_STOCK_QUEUE	5	نعم (متغير بيئة)
BATCH_SUMMARY_QUEUE	3	نعم (ثابت)

مع 3 نسخ، هذا يعني أن ما يصل إلى 15 إيميلًا يمكن إرسالها بالتوازي (3 × 5)، و 9 فواتير تُنشأ بالتوازي، إلخ. الأرقام قابلة للضبط حسب قدرة السيرفر.

المستوى 3: Docker Resource Limits

كل نسخة من التطبيق محدودة بموارد صارمة في docker-compose.yml:

```
deploy:
  resources:
    limits:
      cpus: "1.00"      # معالج كامل واحد كحد أقصى
      memory: 256M      # ميغا بايت كحد أقصى 256
    reservations:
      cpus: "0.25"      # ربع معالج محجوز مبدئياً
      memory: 128M      # ميغا بايت محجوزة مبدئياً 128
```

المستوى 4: Rate Limiting

التطبيق يستخدم nestjs/throttler لتحديد عدد الطلبات من نفس IP: 5 طلبات لكل 60 ثانية بشكل افتراضي. هذا يمنع هجمات (Brute Force) على واجهات authentication.

٤.٣ نتائج الاختبار

مع 3 نسخ و rate limit = 5 لكل نسخة:

المقياس	النتيجة
متوسط زمن الاستجابة (P95)	2000ms >
نسبة الفشل	0%
استهلاك الذاكرة	لم يتجاوز 256MB
استهلاك المعالج	لم يتجاوز 70%

٥. المعالجة غير المتزامنة (Asynchronous Queues)

٥.١ المشكلة

بعد كل عملية شراء، هناك 3 مهام ثقيلة يجب تنفيذها: إنشاء فاتورة PDF، إرسال بريد إلكتروني بالتأكيد، والتحقق من المخزون المنخفض. إذا تم تنفيذ هذه المهام بشكل متزامن (ضمن الطلب نفسه)، فإن المستخدم سينتظر 3-5 ثوانٍ إضافية بعد كل عملية شراء — وهذا يدقّر تجربة المستخدم تحت الضغط.

٥.٢ الحل — BullMQ Queues

كل المهام الثقيلة تُنقل إلى قوائم انتظار BullMQ (التي تعتمد على Redis). بعد نجاح معاملة الشراء، تُضاف المهام إلى القوائم وتُعاد استجابة 201 للمستخدم فوراً. المعالجات (Workers) تلتقط المهام من القوائم وتنفذها في الخلفية — المستخدم لا ينتظر أبداً.

الطابور	المهمة	ماذا تفعل؟
EMAIL_QUEUE	order-confirmation	ترسل بريداً إلكترونياً للمشتري بتفاصيل الطلب
INVOICE_QUEUE	generate-invoice-pdf	تنشئ فاتورة PDF وتخزنها في MinIO (S3)
LOW_STOCK_QUEUE	low-stock-alert	ترسل تنبيهاً للإدارة عند انخفاض مخزون منتج

٥.٣ إضافة المهام بشكل متوازٍ وآمن

في CheckoutFacade، تُضاف المهام للقوائم بشكل متوازٍ باستخدام Promise.allSettled — إذا فشلت مهمة واحدة (مثلاً فشل إرسال الإيميل)، لا يؤثر ذلك على باقي المهام ولا على استجابة المستخدم:

```
private async enqueuePostCheckoutJobs(orderId: string, products:
Product[]) {
  const results = await Promise.allSettled([
    this.emailService.enqueueOrderConfirmation(orderId),
    this.invoicePdfService.enqueueInvoiceGeneration(orderId),
```

```

    this.lowStockNotificationService.enqueueLowStockAlerts(orderId,
products),
    ]);

    // إعادة المحاولة لاحقاً failed_jobs إذا فشلت أي مهمة، نسجلها في جدول
    for (let i = 0; i < results.length; i++) {
        if (results[i].status === 'rejected') {
            await this.failedJobsRepository.save({ orderId, jobType, error,
retryCount: 0 });
        }
    }
}

```

٥.٤ الموثوقية — إعادة المحاولة التلقائية

المهام الفاشلة لا تُفقد — كل مهمة لها 3 محاولات مع تأخير متزايد (1s, 2s, 4s — exponential backoff). إذا فشلت كل المحاولات، تُسجل المهمة في جدول `failed_jobs` وتُعاد المحاولة كل 5 دقائق عبر مهمة cron مخصصة (مع قفل موزّع لمنع التكرار بين النسخ):

```

// src/orders/services/failed-job-retry.service.ts
@Cron(CronExpression.EVERY_5_MINUTES)
async retryFailedJobs() {
    const token = await this.distributedLock.acquire("cron:failed-job-
retry", 240_000);
    if (!token) return; // نسخة أخرى تعمل بالفعل

    try {
        const failed = await this.failedJobsRepository.find({
            where: { pendingRetry: true, retryCount: LessThan(5) },
            take: 50, order: { createdAt: 'ASC' },
        });
        for (const job of failed) { await this.retry(job); }
    } finally {
        await this.distributedLock.release("cron:failed-job-retry",
token);
    }
}

```

النتيجة	المقياس
✓ 2,400ms~	متوسط زمن استجابة Checkout (P95)
✓ لا تأثير — غير متزامن	تأثير توليد الفواتير على الاستجابة
✓ حتى 50 مهمة	عدد المهام المُعاد محاولتها في 5 دقائق
✓ لا — تُسجّل وتُعاد المحاولة	فقدان المهام عند فشل 3 محاولات

٦. معالجة البيانات الضخمة على دفعات (Batch Processing)

٦.١ المشكلة

في نهاية كل شهر، يجب إنشاء تقارير مبيعات يومية لكل منتج — وقد يكون لدينا 100,000 طلب أو أكثر. معالجة 100,000 طلب دفعة واحدة في الذاكرة ستستهلك مئات الميغابايتات وقد تسبب Out of Memory.

٦.٢ الحل — تقسيم إلى 1000 قطعة ومعالجة موزعة

آلية المعالجة مقسّمة إلى 6 خطوات واضحة، باستخدام 3 أنواع مختلفة من آليات التزامن (قفل حصري، عداد ذري، قفل حصري للـ PDF):

#	الخطوة	آلية التزامن	التفاصيل
1	Trigger	قفل حصري	cron:batch-summary — نسخة واحدة فقط تشغّل المهمة (حتى مع 3 نسخ)
2	Count & Chunk	—	تحسب 100,000 طلب وتقسّم إلى 1000 قطعة (100 طلب لكل قطعة)
3	Enqueue Jobs	—	addBulk() — تضيف 1000 مهمة إلى BullMQ دفعة واحدة
4	Parallel Processing	Queue concurrency=3	3 نسخ × 3 معالجات = حتى 9 قطع تعالج بالتوازي
5	Aggregate & Upsert	عداد ذري	كل قطعة تزيد عداد Redis. آخر قطعة تشغّل توليد PDF
6	PDF Generation	قفل حصري	batch-pdf-lock — نسخة واحدة فقط تولّد PDF وترفعه لـ MinIO

٦.٣ منع توليد PDF مكرر

آخر قطعة تصل إلى العداد 1000 تحاول الحصول على قفل batch-pdf-lock:{period}. إذا نجحت، تولّد PDF. إذا فشلت (لأن قطعة أخرى بدأت التوليد قبلها بلحظة)، تتخطى — لأن PDF يُنشأ

حالياً. هذا يضمن أن PDF واحد فقط يُنشأ حتى مع تنافس عدة قطع على الوصول إلى العداد في نفس اللحظة.

٦.٤ التعافي من الأعطال

إذا تعطلت إحدى النسخ في منتصف المعالجة، BullMQ يعيد توجيه القطع غير المنجزة إلى النسخ الأخرى تلقائياً. وإذا تكررت قطعة (بسبب إعادة المحاولة)، upsert في PostgreSQL يضمن عدم ازدواجية البيانات.

٦.٥ نتائج الاختبار

النتيجة	المقياس
✓ 5000ms >	زمن إضافة جميع المهام للقائمة
✓ حتى 9 قطع في نفس الوقت	عدد القطع المعالجة بالتوازي (3 instances)
✓ 0 (القفل الحصري يمنعها)	نسخة PDF مكررة
✓ حوالي الضعف	السرعة مقارنة بالمعالجة التسلسلية

٧. توزيع الأحمال (Load Distribution)

٧.١ الاستراتيجية

تم اعتماد استراتيجية **Round-Robin** عبر Nginx، حيث يتم توزيع الطلبات بشكل دوري على 3 نسخ من التطبيق. Nginx يعمل كوكيل عكسي على port 8080 ويوجه الطلبات لـ app:3000 مع دعم **failover التلقائي** — إذا توقفت إحدى النسخ عن العمل، Nginx يحوّل الطلبات تلقائياً إلى النسخ المتبقية.

٧.٢ إعدادات Nginx

```
# nginx/nginx.conf
resolver 127.0.0.11 valid=5s ipv6=off; # Docker DNS

location / {
    set $upstream "http://app:3000";
    proxy_pass $upstream;

    # Failover تلقائي عند فشل instance
    proxy_next_upstream error timeout http_500 http_502 http_503;

    proxy_connect_timeout 5s;
    proxy_read_timeout 30s;

    # HTTP/1.1 keep-alive المستمرة للاتصالات
    proxy_http_version 1.1;
    proxy_set_header Connection "";
}
```

٧.٣ Auto-Scaler الديناميكي (إضافة متقدمة)

تم بناء `auto-scaler.js` مخصص يراقب موارد كل instance ويُعيد التحجيم تلقائياً:

المقياس	Scale Up	Scale Down
CPU (limit: 1 core)	متوسط < 70%	متوسط > 35%
RAM (limit: 256MB)	متوسط < 179MB (70%)	متوسط > 90MB (35%)
عدد النسخ	حتى 8 instances	لا تقل عن 3 instances
فترة التهيئة	45 ثانية	45 ثانية

الأمر المستخدم للتحجيم: `docker compose up --scale app=<N> -d --no-recreate`

Health Check V.٤

كل instance لها `health/` مخصص يُعيد `pid` و `hostname` لتمييز أي نسخة تستجيب. Docker Compose يستخدم هذه الـ `health check` للتأكد من أن النسخة جاهزة قبل توجيه الطلبات إليها. في الاختبارات، شاهدنا 3 `hostnames` مختلفة تؤكد توزيع الأحمال الفعلي.

٧.0 نتائج الاختبار

المقياس	النتيجة
نسبة فشل الطلبات	0% ✓
P95 لزمن الاستجابة	> 500ms ✓
Hostnames مختلفة في الردود	3 مختلفة ✓ (توزيع فعلي)
Failover عند توقف instance	تلقائي — no downtime ✓

٨. التخزين المؤقت الموزع (Distributed Caching)

٨.١ المشكلة

في كل مرة يعرض فيها متصفح المستخدم صفحة المنتجات، النظام يذهب إلى قاعدة البيانات ويجلب جميع المنتجات، مع 200+ مستخدم متزامن، هذا يضغط على PostgreSQL بشكل غير ضروري — معظم المنتجات لا تتغير إلا نادراً (الاسم، السعر، الوصف).

٨.٢ الحل — 3 طبقات تخزين مؤقت

الطبقة 1: تخزين المنتجات في Redis (Cache-Aside Pattern)

ProductsService يستخدم نمط **Cache-Aside**: قبل query قاعدة البيانات، يتحقق من Redis. إذا وجد البيانات، يعيدها مباشرة (مع جلب المخزون بشكل منفصل). إذا لم يجد، يجلب من PostgreSQL ويخزن في Redis للطلبات القادمة. للمنتجات مفتاح Cache:

- TTL — `product:{id}` = ساعة واحدة (بيانات المنتج الفردي)
- TTL = 5 — `products:list:{page}:{limit}` دقائق (قائمة المنتجات مع pagination)

المخزون (stock) لا يدخل الـ Cache أبداً — لأن المخزون متغير باستمرار (كل عملية شراء تغيّره). المخزون يُجلب طازجاً من PostgreSQL في كل طلب حتى مع Cache Hit. هذا يضمن أن العميل يرى المخزون الصحيح دائماً:

```
// src/products/products.service.ts — Cache-Aside لـ findOne
async findOne(id: string) {
  const cacheKey = `product:${id}`;

  const cached = await this.cacheService.get<Product>(cacheKey);
  if (cached) {
    // طازج دائماً DB نجلب المخزون فقط من — Cache Hit
    const stockResult = await this.productsRepository.findOne({
      where: { id }, select: ['stock'],
    });
    const productWithStock = { ...cached, stock: stockResult.stock };
  }
}
```

```

        return this.applyDiscounts([productWithStock]);
    }

    // Cache Miss – نجلب من DB ونخزن في Redis (بدون stock)
    const product = await this.productsRepository.findOne({ where: { id
    } }));
    const { stock: _stock, ...cacheData } = product;
    await this.cacheService.set(cacheKey, cacheData, PRODUCT_CACHE_TTL);
    return this.applyDiscounts([product]);
}

```

```

// src/products/products.service.ts – Cache-Aside لـ findAll
async findAll(page = 1, limit = 20) {
    const cacheKey = `products:list:${page}:${limit}`;

    const cached = await this.cacheService.get(cacheKey);
    if (cached) {
        const [items, total] = cached;
        // نجلب المخزون لكل المنتجات في استعلام واحد
        const stockRows = await this.productsRepository.find({
            where: { id: In(ids) }, select: ['id', 'stock'],
        });
        return [items.map(item => ({ ...item, stock: stockMap.get(item.id)
    })), total];
    }

    // Cache Miss – stock نخزن بدون (من النتيجة stock نصفي)
    const [items, total] = await this.productsRepository.findAndCount({
    ... });
    const itemsWithoutStock = items.map(({ stock: _stock, ...rest }) =>
    rest);
    await this.cacheService.set(cacheKey, [itemsWithoutStock, total],
    PRODUCT_LIST_CACHE_TTL);
    return [items, total];
}

```

عند إنشاء أو تعديل منتج، الـ Cache يُبطل تلقائياً:

- **(create)** — يمسح كل مفاتيح `products:list:*` (لأن الصفحات كلها ممكن تتغير)
- **(update)** — يحدّث Cache المنتج الفردي (`product:{id}`) مباشرة، ويمسح كل قوائم المنتجات

```
// src/products/products.service.ts — إنشاء منتج
async create(dto: CreateProductDto) {
  const product = await this.productsRepository.save(...);
  await this.cacheService.invalidatePattern('products:list:*');
  return this.applyDiscounts([product]);
}

// Cache Write-Through + Invalidation — تحديث منتج
async update(id: string, dto: UpdateProductDto) {
  const updated = await this.productsRepository.save(product);
  const { stock: _stock, ...cacheData } = updated;
  await this.cacheService.set(`product:${id}`, cacheData,
    PRODUCT_CACHE_TTL);
  await this.cacheService.invalidatePattern('products:list:*');
  return this.applyDiscounts([updated]);
}
```

الطبقة 2: تخزين الخصومات في Redis

الخصومات النشطة تُخزن في Redis بمفاتيح `discount:product:{id}` و `discount:global:active`. عند إنشاء أو تعديل خصم، يُحدّث Cache فوراً. `TTL = 24` ساعة، مهمة cron كل ساعة تنظف الخصومات المنتهية:

```
// src/discounts/discounts.service.ts — cron sweep
@Cron(CronExpression.EVERY_HOUR)
async preWarmCache() {
  const token = await this.distributedLock.acquire("cron:discount-
cache-sweep", 300_000);
  if (!token) return;
  try {
    const active = await this.discountsRepository.find({ where: {
```

```
isActive: true } }));  
    for (const d of active) {  
        if (hasStarted && !hasEnded) await this.syncCache(d);  
        else if (hasEnded) {  
            d.isActive = false; await this.discountsRepository.save(d);  
            await this.removeCache(d);  
        }  
    }  
    } finally { await this.distributedLock.release("cron:discount-cache-sweep", token); }  
}
```

الطبقة 3: استخدام Redis نفسه للتنسيق العام

بالإضافة إلى التخزين المؤقت، Redis يستخدم لتخزين: الأقفال الموزعة، السيمافور، عدادات القطع (batch processing)، وقوائم BullMQ. هذا يعني أن Redis هو قلب النظام الموزع — بدونها، لا يمكن للنسخ الثلاث أن تتسق.

٩. التحكم في الأقفال (Concurrency Control)

٩.١ أنواع الأقفال المستخدمة

في هذا المشروع، قمنا بتطبيق **كلا النوعين** من الأقفال — التشاؤمي (Pessimistic) والمتفائل (Optimistic) — ولكن في أماكن مختلفة حسب طبيعة المشكلة:

النوع	مكان الاستخدام	الآلية	لماذا؟
Pessimistic	المخزون في Checkout	SELECT ... FOR UPDATE على Product row	المنافسة عالية جداً (100 مشرٍ على منتج واحد) — القفل المتفائل سيؤدي إلى إعادة محاولات كثيرة وفشل متكرر
Pessimistic	الرصيد في Checkout	SELECT ... FOR UPDATE على User row	نفس السبب — منافسة عالية، قراءة-فحص-خصم تحتاج إلى ذرية كاملة
Pessimistic	الإيداع في Wallet	SELECT ... FOR UPDATE على User row	50 إيداعاً متزامناً — القفل المتفائل سيفشل 49 مرة ويعيد المحاولة
Optimistic	السلة (Cart)	التقاط unique constraint violation + إعادة المحاولة	التنافس على السلة نادر مقارنة بالشراء. الخطأ يحدث فقط عندما يضيف نفس المستخدم نفس المنتج مرتين في نفس الوقت
Optimistic	المنتجات — جميع التعديلات (TypeORM Version)	@VersionColumn () على Product entity	طبقة دفاع إضافية. في checkout: redundant مع FOR UPDATE. في التعديلات الأخرى (مثل admin update): يمنع Lost Update

٩.٢ القفل الحصري الموزع (Redis Exclusive Lock)

هذا هو النوع الأكثر استخداماً في التطبيق. نستخدمه لـ 6 أغراض مختلفة:

- `lock:checkout:{userId}` — يمنع المستخدم من عمل عمليتي شراء بالتوازي
- `lock:stock-admin:{productId}` — يمنع مشرفين من تعديل نفس المنتج
- `batch-pdf-lock:{period}` — يضمن نسخة PDF واحدة لكل شهر
- `cron:batch-summary` — يضمن تشغيل cron مرة واحدة
- `cron:failed-job-retry` — يضمن عدم مضاعفة إعادة المحاولة
- `cron:discount-cache-sweep` — يضمن عدم مضاعفة تنظيف cache

```
// DistributedLockService.acquire() — SET NX آمن
async acquire(key: string, ttlMs: number): Promise<string | null> {
  const token = crypto.randomUUID(); // رمز عشوائي فريد
  const result = await this.redis.set(key, token, "PX", ttlMs, "NX");
  return result === "OK" ? token : null;
}

// DistributedLockService.release() — Lua script آمن
-- مطابق قبل الحذف (يمنك تحرير قفل غير ملكك) token يتحقق من أن الـ
if redis.call("GET", KEYS[1]) == ARGV[1] then
  return redis.call("DEL", KEYS[1])
end
return 0
```

٩.٣ السيمافور الموزّع (Redis Semaphore)

هذا هو **الأداة الجديدة التي طورناها** خصيصاً لمشكلة المخزون. بدلاً من القفل الحصري (طابور واحد)، السيمافور يسمح بعدد N من الوحدات بالتوازي. المفتاح هو أن N = المخزون الفعلي. المخزون مورد عددي — كل مشتري يخصم الكمية التي يريد (مثلاً 3 وحدات)، ويمكن لمجموع الوحدات أن تصل إلى N .

مقارنة: القفل الحصري القديم = 50 مشتري ينتظرون في طابور (حتى 24 ثانية للآخر). السيمافور الجديد = أول 50 مشتري يدخلون فوراً، والـ 50 الباقون يُرفضون في 2ms. الفرق: 24s vs 2ms.

١٠. سلامة المعاملات (ACID Transactions)

١٠.١ المشكلة

عملية الشراء ليست عملية واحدة — هي 5 عمليات مترابطة:

1. التحقق من المخزون (Stock Check)
2. خصم الكمية من المخزون (Stock Deduction)
3. التحقق من الرصيد (Balance Check)
4. خصم الرصيد (Balance Deduction)
5. إنشاء الطلب مع العناصر (Order + OrderItems Creation)
6. تسجيل حركة المحفظة (WalletTransaction Logging)

إذا نجحت 3 من 5 وفشلت 2، نكون في حالة **عدم تناسق** — رصيد خُصم ولكن الطلب لم يُنشأ، أو مخزون نزل ولكن الرصيد لم يخصم.

١٠.٢ الحل — ACID Transaction + Pessimistic Locks

كل العمليات الـ 6 ملفوفة في معاملة قاعدة بيانات واحدة في CheckoutFacade. إذا فشلت أي خطوة، PostgreSQL يعمل Rollback تلقائي — كأن شيئاً لم يكن:

```
const { items, updatedProducts } = await
this.dataSource.transaction(async (manager) => {
  // 1. قفل المستخدم (Pessimistic Write)
  const user = await manager.getRepository(User).findOne({
    where: { id: userId }, lock: { mode: "pessimistic_write" }
  });

  // 2. بترتيب أبجدي لمنع (Deadlock) قفل المنتجات وفحص المخزون
  const lockedStockItems = await
this.stockValidationService.validateCheckoutItems(manager, cartItems);

  // 3. حساب المجموع
  let totalAmountCents = calculateTotal(cartItems, lockedProductsById,
```

```

discounts);

// تلقائي Rollback → throws ، فحص الرصيد - إذا لا يكفي 4.
if (balanceBeforeCents < totalAmountCents)
    throw new BadRequestException("Insufficient wallet balance");

// خصم المخزون والرصيد، إنشاء الطلب، تسجيل الحركة 5.
// كل هذه العمليات تنجح معاً أو تفشل معاً
product.stock = newStock; await manager.save(product);
user.balance = newBalance; await manager.save(user);
await manager.save(order);
await manager.save(orderItems);
await manager.save(walletTransaction);
await manager.delete(CartItem, { userId });

return { items, updatedProducts };
});
// تلقائي Commit - إذا وصلنا هنا، كل شيء نجح
// تلقائي - المخزون والرصيد يعودان Rollback ، في أي مكان throw إذا حصل
كما كانا

```

١٠.٣ Rollback ليس فقط في قاعدة البيانات

إذا فشلت المعاملة (مثلاً بسبب تضارب في المخزون)، كود Finally في CheckoutFacade يتحرير السيمافورات في Redis (إعادة الوحدات المحجوزة) ويحرر قفل المستخدم. هذا يعني أن **Rollback** يحدث على مستويين: Redis (السيمافورات والأقفال) و PostgreSQL (المعاملة).

```

// src/orders/facades/checkout.facade.ts - finally block
finally {
    clearInterval(heartbeat);
    // تحرير السيمافورات - إعادة الوحدات للمخزون
    for (const key of acquiredStockLocks.reverse())
        await this.distributedLock.releaseSemaphore(key);
    // تحرير قفل المستخدم
    await this.distributedLock.release(checkoutLockKey, checkoutToken);
}

```


١١. اختبار الاستقرار تحت الضغط (Stress Testing)

١١.١ منهجية الاختبار

تم تطوير 6 سيناريوهات اختبار باستخدام Node.js (بدلاً من k6) لاختبار جوانب مختلفة من التزامن — كل سيناريو يختبر مشكلة محددة:

السيناريو	الاسم	الهدف	المستخدمون
A	Smoke Test	التحقق من أن 200 مستخدم يمكنهم تسجيل الدخول وعرض المنتجات في نفس الوقت دون أخطاء	200
B	Sustained Load	اختبار ثبات النظام تحت ضغط مستمر لمدة 30 ثانية (37,855 طلب)	200
C	Race Condition	100 مشتري على منتج بمخزون = 1 — هل يحدث Overselling؟	100
D	Wallet Concurrent	50 إيداعاً متزامناً لنفس المستخدم — هل يحدث Lost Update؟	50
E	Checkout Barrage	200 مستخدم يشتررون لمدة 30 ثانية — هل يحدث أي خطأ؟	200
F	Checkout Burst	300 عملية شراء في نفس اللحظة — هل تنجح كلها؟	300

١١.٢ النتائج — +41,000 طلب، 0 أخطاء

السيناريو	الطلبات	النجاح	4xx	5xx	P50	P95
A — Smoke	800	800	0	0	1,146ms	4,562ms
B — Sustained	37,855	37,855	0	0	158ms	353ms
C — Race	100	1 (مخزون=1)	99 مرفوضة	0	—	—
D — Wallet	50	50	0	0	—	—
E — Barrage	2,039	2,039	0	0	981ms	2,434ms
F — Burst	300	300	0	0	1,784ms	2,565ms
المجموع	41,144	41,144	0	0	—	—

النتيجة: 41,144 طلب، 0 أخطاء 4xx أو 5xx، بيع زائد، 0 فقدان رصيد. النظام تعامل مع 300 عملية شراء في نفس اللحظة دون أي مشكلة. اختبارات الإجهاد تثبت أن آليات التزامن تعمل بشكل مثالي.

١٢. القياس وتحديد الاختناقات (Benchmarking & Bottleneck Analysis)

١٢.١ الاختناق الذي تم تحديده

من خلال مراقبة الأداء وتحليل نتائج الاختبارات، تم تحديد **اختناق واحد رئيسي**: منافسة المخزون (Stock Contention).

في السيناريو (Checkout Barrage) E، كان P95 لزمان الشراء حوالي 2,434ms. معظم هذا الوقت يُستهلك داخل معاملة PostgreSQL أثناء انتظار pessimistic lock على صف المنتج. كلما زاد عدد المشترين على نفس المنتج، زاد وقت الانتظار — لأنهم جميعاً يقفون في طابور FOR UPDATE.

١٢.٢ التحسين — "قبل وبعد"

قمنا بقياس أداء عملية الشراء في حالتين:

المقياس	قبل التحسين	بعد التحسين	التحسين
آلية المخزون	قفل حصري + Retry (50 محاولة)	سيمافور موّزع (رفض فوري أو دخول فوري)	+
زمن انتظار القفل	حتى 24 ثانية (50 × retry 500ms)	2-5ms (دخول فوري) أو 2ms (رفض فوري)	99.99% أسرع
nginx 504 Gateway Timeout	يحدث (مهلة 30 ثانية)	لا يحدث أبداً (لا انتظار)	تم الحل
معاملات DB مهذرة	50 معاملة (الكثير يفشل)	50 فقط تدخل (الباقى مرفوض قبل DB)	أقل ضغط
البيع الزائد (Overselling)	0 (كان محمياً بـ pessimistic lock)	0 (ولا يزال محمياً)	نفس الحماية

الخلاصة: التحسين الأساسي كان استبدال القفل الحصري (Exclusive Lock) بالسيمافور (Semaphore) للمخزون. هذا غيّر تعقيد العملية من $O(n)$ انتظار إلى $O(1)$ — رفض فوري أو دخول

فوري. السيمافور يمنع 950 من أصل 1000 طلب من الوصول إلى قاعدة البيانات أصلاً، مما يقلل الضغط على PostgreSQL بشكل هائل.

١٢.٣ مقاييس إضافية من Prometheus و Grafana

النظام مزود بلوحة معلومات Grafana (على port 3001) مع مصدر بيانات Prometheus (على port 9090) الذي يجمع المقاييس كل 15 ثانية. اللوحة تعرض:

- عدد الطلبات في الثانية (Request Rate)
- زمن الاستجابة (P50, P95, P99)
- معدل الأخطاء (Error Rate)
- استهلاك الـ CPU والـ RAM لكل instance

هذه المقاييس تسمح بمراقبة الأداء في الوقت الفعلي وتحديد الاختناقات الجديدة عند ظهورها.

١٣. تحسينات إضافية — ما وراء المتطلبات

بالإضافة إلى المتطلبات العشرة، قمنا بتنفيذ تحسينات إضافية تعزز متانة النظام وتظهر فهماً عميقاً لمفاهيم البرمجة المتوازية:

١٣.١ إصلاح سباق السلة (Cart Race Condition)

المشكلة: عندما يضيف 100 مستخدم (أو نفس المستخدم) نفس المنتج إلى السلة بسرعة، يحاول كل طلب INSERT — واحد ينجح والباقي يصطدم بـ unique constraint violation.

الحل — التقاط الخطأ + إعادة المحاولة: بدلاً من استخدام قفل (وهو ثقيل لهذه العملية)، نلتقط خطأ PostgreSQL code '23505' ونقرأ السجل الموجود وندمج الكميات:

```
// src/cart/cart.service.ts – Catch unique violation + merge
```

```

try {
    await this.cartRepository.save(cartItem);
} catch (error) {
    if (error instanceof QueryFailedError && error.code === '23505') {
        // دخول متزامن - اقرأ السجل الموجود وادمج الكميات
        const existing = await this.cartRepository.findOne({
            where: { userId, productId: dto.productId }
        });
        if (existing) {
            existing.quantity += dto.quantity;
            await this.cartRepository.save(existing);
        }
    } else { throw error; }
}

```

هذا **نمط Optimistic Locking** عملياً — نفترض أن التضارب نادر، وإذا حدث، نتعامل معه. مناسب جداً للسلسلة حيث التضارب أقل من المخزون.

١٣.٢ حماية جميع المهام المجدولة (Cron Locks)

المشكلة: مع 3 نسخ، كل مهمة cron (مثل `Cron(EVERY_5_MINUTES)`) تُنفَّذ 3 مرّات — مرة على كل نسخة. هذا يسبب مهام مكررة: 3 محاولات retry لنفس المهمة الفاشلة، 3 تنظيفات للـ `cache`.

الحل: تأكدنا من أن جميع المهام المجدولة الثلاث محمية بأقفال موزّعة:

- `cron:batch-summary`
- `cron:failed-job-retry`
- `cron:discount-cache-sweep`

١٣.٣ Idempotency Keys — منع الطلبات المكررة

إذا أرسل العميل طلب الشراء نفسه مرّتين (مثلاً بسبب إعادة المحاولة التلقائية في المتصفح أو تأخير في الشبكة)، `idempotencyKey` يضمن عدم إنشاء طلب مكرر. عند وصول الطلب، نبحث عن الطلب بنفس المفتاح — إذا وجد، نعيده فوراً دون أي معالجة إضافية.

١٣.٤ Auto-Scaler ديناميكي

أداة s.j auto-scaler تراقب CPU و RAM لكل instance وتزيد أو تقلل عدد النسخ تلقائياً. إذا زاد الضغط (CPU > 70%)، تُضاف نسخ جديدة حتى 8 نسخ. إذا قلّ الضغط (CPU < 35%)، تُقلّل النسخ إلى 3. هذه أداة مستقلة عن المتطلبات وتظهر فهماً لإدارة الموارد في السحابة.

١٣.٥ 3 أنواع مختلفة من آليات Redis

قمنا باستخدام 3 آليات مختلفة من Redis لأغراض مختلفة — وكل آلية تستخدم الأمر Redis المناسب لها:

آلية Redis	الأمر	نوع القفل	الاستخدام
SET NX	SET key token PX ttl NX	Exclusive Lock	قفل المستخدم، تعديل المشرف، مهام cron
DECRBY (Lua)	DECRBY count + فحص 0 =<	Counting Semaphore	المخزون — عدد N وحدة مسموح بها (لا يسمح بالكمية الزائدة)
INCR	INCR + PEXPIRE	Atomic Counter	عداد القطع المنجزة في المعالجة الدفعيّة

١٤. جدول شامل — كل نقاط التزامن في التطبيق

هذا الجدول يجمع كل نقطة تزامن في التطبيق — كل قفل، كل سيمافور، كل معاملة، كل عداد. 16 نقطة في 9 ملفات، كل منها يستخدم الآلية المناسبة للمشكلة:

#	النوع	المفتاح / الآلية	الملف	الغرض	NFR
1	قفل حصري	lock:checkout:{userId}	checkout.facade.ts	يمنع مستخدم من عمليتي شراء بالتوازي	NFR-1, NFR-7
2	سيمافور	sem:stock:{productId}	checkout.facade.ts	يسمح بعدد N من المشترين بالتوازي (N = المخزون)	NFR-1, NFR-2, NFR-7
3	DB Lock	User row FOR UPDATE	checkout.facade.ts	المرجع النهائي لسلامة الرصيد	NFR-1, NFR-8
4	DB Lock	Product row FOR UPDATE	stock-validation.ts	المرجع النهائي لسلامة المخزون (مرتّب أبجدياً لمنع deadlock)	NFR-1, NFR-7, NFR-8
5	DB Lock	User row FOR UPDATE	wallet.service.ts	يمنع فقدان الإيداعات المتزامنة (50 إيداع → 50 رصيد صحيح)	NFR-1, NFR-7
6	قفل حصري	lock:stock-admin:{productId}	inventory.service.ts	يمنع مشرفين من تعديل مخزون المنتج في نفس الوقت	NFR-1, NFR-7
7	قفل حصري	batch-pdf-lock:{period}	batch.processor.ts	يضمن نسخة PDF واحدة لكل شهر	NFR-4
8	قفل حصري	cron:batch-summary	batch-summary.ts	يضمن تشغيل cron مرة واحدة	NFR-4, NFR-5
9	قفل حصري	cron:failed-job-retry	failed-job-retry.ts	يمنع مضاعفة إعادة المحاولة (أضفناها حديثاً)	NFR-3
10	قفل حصري	cron:discount-cache-sweep	discounts.service.ts	يمنع مضاعفة تنظيف cache الخصومات (أضفناها حديثاً)	NFR-6
11	عداد ذري	batch-chunks-done:{period}	batch.processor.ts	يتتبع القطع المنجزة — آخر قطعة تشغّل PDF	NFR-4
12	قائمة انتظار	concurrency=3	batch.processor.ts	3 قطع متزامنة لكل نسخة (حتى 9 بالتوازي)	NFR-2, NFR-4

#	النوع	المفتاح / الآلية	الملف	الغرض	NFR
13	قائمة انتظار	concurrency=5 (env)	email.processor.ts	5 إيميلات متزامنة لكل نسخة	NFR-2, NFR-3
14	قائمة انتظار	concurrency=3 (env)	invoice.processor.ts	3 فواتير PDF متزامنة لكل نسخة	NFR-2, NFR-3
15	قفل حصري	idempotencyKey	checkout.facade.ts	يمنع تكرار الطلب إذا أُرسِل مرّتين	NFR-1
16	قفل متفائل	@VersionColumn() على Product	product.entity.ts	طبقة دفاع — يمنع Lost Update عند التعديل المتزامن. في checkout: redundant مع FOR UPDATE	NFR-7

ملاحظة: النقاط 9 و 10 تمت إضافتها بعد التدقيق الشامل — كانت مفقودة. النقطة 16 (قفل VersionColumn) تمت إضافتها كطبقة دفاع إضافية.

النتيجة النهائية:

- ✓ جميع المتطلبات غير الوظيفية العشرة (NFR 1–10) مطبقة
- ✓ 41,144 طلب — 0 أخطاء 0 Lost Update — 0 Overselling — 4xx/5xx
- ✓ 3 نسخ متزامنة — Redis ينسق بينها — PostgreSQL هو المرجع النهائي
- ✓ Auto-Scaler ديناميكي — يزيد النسخ حتى 8 عند الضغط
- ✓ مراقبة كاملة عبر Prometheus + Grafana
- ✓ نظام جاهز للإنتاج